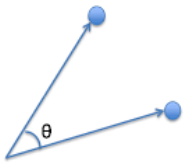
이번 과제 프로젝트의 목표는 OpenAI 의 Embeddings API 를 활용하여 의미 기반 도서 추천 시스템을 구축하는 것이다. 기존의 검색 시스템은 주로 키워드 매칭 방식에 의존하며, 사용자가 입력한 단어가 정확히 포함된 결과만을 반환하는 한계를 가진다. 예를 들어 특정 질의를 입력했을 때, 해당 단어가 정확히 포함되지 않은 경우 의미적으로 유사한 책임에도 검색되지 않는 문제가 발생한다. 이러한 한계를 해결하기 위해 본 프로젝트에서는 텍스트의 의미를 벡터로 표현하는 임베딩(Embedding) 기법을 활용하였다. 이를 통해 단순한 문자열 비교가 아닌, 의미적 유사성을 기반으로 도서를 추천할 수 있도록 시스템을 설계하였다. 또한, 최근 인공지능에서 중요한 개념인 "검색과 분류는 동일한 문제"라는 통찰을 직접 구현해보고자 하였다. 이를 위해 추천 시스템뿐만 아니라 제로샷 분류 기능도 함께 구현하여, 동일한 벡터 공간에서 다양한 문제를 해결할 수 있음을 확인하였다.

먼저 임베딩이란, 텍스트 데이터를 숫자로 이루어진 벡터로 변환하는 기술이다. 예를 들어, "사랑 이야기"와 "로맨스 소설"은 단어는 다르지만 의미가 비슷하므로, 임베딩 공간에서는 서로 가까운 위치에 배치된다. 이러한 문제를 해결하기 위해 본 프로젝트에서는 OpenAI 의 text-embedding-3-small 모델을 사용하여 각 텍스트를 고차원 벡터로 변환하였다.
벡터 공간은 임베딩된 데이터는 벡터 공간 상의 하나의 점으로 표현된다. 이 공간에서는 두 벡터 간의 거리를 통해 의미적 유사성을 측정할 수 있다. 즉, 벡터 간 거리가 가까울수록 두 텍스트의 의미가 유사하다고 판단할 수 있다.
코사인 유사도는 두 벡터의 방향이 얼마나 유사한지를 측정하는 방법이다. 다음과 같은 수식으로 정의된다.

$$sim(A,B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$



여기서 A 와 B 는 두 벡터이며, 값이 1 에 가까울수록 두 벡터가 유사함을 의미한다. 코사인 값이 크면, 사잇각은 작아지고, 유사도는 높아진다. 본 프로젝트에서는 이 공식을 직접 구현하여 사용하는 방식으로 적용하였다.

시스템의 전체 구조는 수업에서 배운 3 단계 프로세스인 텍스트 전처리 및 확장(gpt-4.1-mini) -> 임베딩 생성(text-embedding-3-small) -> 유사도 계산 및 결과 반환 단계로 설계하였다.

데이터의 흐름은 먼저 사용자가 자연어 질의를 입력하고 gpt-4.1-mini 를 통해 질의를 더 풍부한 문장으로 확장한다. 다음으로 확장된 텍스트를 임베딩으로 변환하고 데이터셋의 각 도서 임베딩과 코사인 유사도를 계산한다. 마지막으로 유사도가 가장 높은 도서를 추천하는 과정이다.

핵심 아이디어는 모든 텍스트 데이터를 벡터 공간(Vector Space) 상의 점으로 변환한 뒤, 이들 간의 거리를 기반으로 의미적 유사성을 판단하는 것이다. 즉, 책의 제목, 설명, 리뷰와 같은 텍스트는 단순한 문자열이 아니라, 고차원의 숫자 벡터로 표현되며, 이 벡터들은 하나의 공통된 공간에 배치된다. 이러한 벡터 공간에서 두 텍스트 간의 의미적 유사성은 코사인 유사도(Cosine Similarity)를 통해 계산된다. 이 개념을 도서 추천 문제에 적용하면 다음과 같다. 사용자가 입력한

질의 문장 역시 동일한 방식으로 임베딩 벡터로 변환되고, 이 벡터와 데이터셋에 존재하는 각 도서의 벡터 간 코사인 유사도를 계산한다. 이후 유사도가 가장 높은 도서를 선택함으로써, 사용자의 의도와 의미적으로 가장 가까운 결과를 반환하게 된다.
이 과정은 수학적으로 보면 "벡터 공간에서 가장 가까운 이웃(Nearest Neighbor)을 찾는 문제"로 해석할 수 있다. 즉, 모든 데이터가 하나의 공간에 점으로 존재하고, 그 중에서 특정 점(질의)에 가장 가까운 점들을 찾는 방식이다. 또한 이 구조는 단순한 추천 시스템뿐 만 아니라 분류 문제에도 동일하게 적용될 수 있다. 예를 들어, 각 장르를 하나의 텍스트 문장으로 정의하고 이를 임베딩 벡터로 변환하면, 새로운 입력 텍스트를 가장 가까운 장르 벡터에 매핑함으로써 자동으로 분류할 수 있다. 이는 별도의 학습 과정 없이도 가능한 제로샷 분류(Zero-shot Classification) 방식이다.
결과적으로, 검색(가장 유사한 데이터 찾기)과 분류(가장 가까운 레이블 찾기)는 모두 동일하게 벡터 간 거리 계산 문제로 통합될 수 있으며, 이 프로젝트에서는 이를 실제 시스템으로 구현했다는 점에서 의의가 있다. 이러한 접근 방식은 기존의 키워드 기반 방법과 달리, 단어의 표면적 일치 여부가 아니라 문장의 의미 자체를 이해하고 처리할 수 있다는 장점을 가진다.

요약하자면 본 시스템은 텍스트를 벡터 공간에 매핑한 뒤 코사인 유사도를 기반으로 최근접 이웃을 탐색하는 방식으로 동작하며, 이를 통해 검색과 분류가 동일한 구조를 가진다는 점을 확인할 수 있었으며, 이 과정은 결국 **"벡터 공간에서 가장 가까운 이웃을 찾는 문제"**로 볼 수 있으며, 이는 검색과 분류가 동일한 구조를 가진다는 점을 보여준다.

데이터셋 구성은 총 50 개 이상의 도서 정보를 포함하도록 구성하였다. 각 데이터는 다음과 같은 필드를 가지며, 또한 gpt-4.1-mini 모델을 활용하여 데이터 일부를 생성 및 보완하였다

```python
def generate_books(n=50):
    books = []
    titles = set()  # 제목 중복 체크

    prompt = """
    책 데이터를 JSON 형식으로 만들어줘.
    반드시 아래 형식만 사용:

    {
      "title": "책 제목",
      "description": "책 설명",
      "review": "리뷰"
    }

    다른 말 절대 하지마.
    절대 중복되지 않게 만들어줘.
    리뷰에 대해서는 인물이나 감상을 더 풍부하게 표현해줘.
    """

    while len(books) < n:
        res = client.chat.completions.create(
            model="gpt-4.1-mini",
            messages=[{"role": "user", "content": prompt}]
        )
```

임베딩 생성 및 캐싱부분은 각 도서의 텍스트는 "제목 + 설명 + 리뷰"로 구성하였다 이 텍스트를 gpt-4.1-mini 를 통해 확장한 후, text-embedding-3-small 모델로 임베딩을 생성하였다.

생성된 임베딩은 embeddings.json 파일로 저장하여, 매번 API 를 호출하지 않도록 캐싱 전략을 적용하였다. 해당 부분에 대한 코드는 아래와 같다.

```python
def get_embedding(text):
    res = client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return res.data[0].embedding
```

```python
def create_embeddings(books):
    for book in books:

        if not all(k in book for k in ["title", "description", "review"]):
            continue

        text = f"{book['title']}. {book['description']}. {book['review']}"

        enriched = enrich_text(text)  # ⭐ 풍부한 임베딩
        book["embedding"] = get_embedding(enriched)

    with open("embeddings.json", "w", encoding="utf-8") as f:
        json.dump(books, f, ensure_ascii=False, indent=2)
```

코사인 유사도는 직접 구현하였으며, sklearn 라이브러리의 cosine_similarity 함수와 결과를 비교하여 정확성을 검증하였다. 코사인 유사도 검증 과정으로는 동일한 두 벡터에 대해 직접 구현한 함수 (cosine_similarity_manual)와 sklearn.metrics.pairwise 라이브러리의 cosine_similarity 함수를 사용하여 유사도를 계산하였다. 검증 과정은 먼저 데이터셋에 포함된 두 개의 도서 임베딩 벡터를 선택한 뒤에 이후 동일한 입력 벡터 쌍에 대해 두 방법으로 각각 코사인 유사도를 계산한다.

```python
# ========================================
# 📌 코사인 유사도 직접 구현
# ========================================
def cosine_similarity_manual(a, b):
    a = np.array(a)
    b = np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# ========================================
# 📌 검증 코드
# ========================================
def verify_cosine(a, b):
    manual = cosine_similarity_manual(a, b)
    lib = cosine_similarity([a], [b])[0][0]
    return manual, lib
```

이때 manual 은 직접 구현한 함수의 결과이며, lib 는 검증을 위한 라이브러리 함수의 결과이다.

실험 결과, 두 값은 소수점 이하에서 매우 근접하거나 같은 값을 보이며, 이는 직접 구현한 코사인 유사도 함수가 수학적으로 올바르게 동작함을 의미하는 것을 알 수 있었다.



이처럼 두 값이 거의 동일하게 나타나는 이유는 두 방식이 동일한 수식을 기반으로 계산되기 때문이다. 다만, 미세한 차이가 발생하는 경우는 부동소수점 연산 과정에서의 오차 때문이며, 이는 일반적인 수치 계산에서 허용되는 범위이다. 이 검증 과정을 통해, 본 프로젝트에서 사용한 코사인 유사도 계산 방식이 신뢰할 수 있음을 확인하였다.

프로젝트의 확장 기능으로는 아래의 네 가지를 구현하였다.

1. 키워드 검색 vs 시맨틱 검색 비교
   → 키워드 기반 검색과 의미 기반 검색의 차이를 확인
2. 제로샷 분류
   → 장르 문장을 임베딩하여 새로운 텍스트를 자동 분류
3. 텍스트 확장 (Enriched Embedding)
   → gpt-4.1-mini 를 활용하여 임베딩 품질 향상
4. Streamlit UI
   → 사용자 친화적인 웹 인터페이스 제공

실험 결과 본 프로젝트에서는 키워드 기반 검색과 시맨틱 검색의 성능 차이를 비교하기 위해 동일한 질의에 대해 두 방식의 결과를 분석하였다.
"슬픈 사랑 이야기" 키워드 기반 검색 결과 키워드 검색은 사용자가 입력한 문자열이 데이터셋의 텍스트(설명, 리뷰 등)에 그대로 포함되는 경우에만 결과를 반환한다.
실험 결과, 위 질의에 대해 일부 도서만 검색되거나 경우에 따라 결과가 전혀 반환되지 않는 경우도 발생하였다 이는 데이터셋에 "슬픈", "사랑"이라는 단어가 정확히 포함되지 않은 경우, 의미적으로 유사한 도서임에도 검색되지 않기 때문이다.

시맨틱 검색 결과 시맨틱 검색에서는 질의를 임베딩 벡터로 변환한 후, 각 도서의 임베딩과 코사인 유사도를 계산하여 결과를 도출한다. 실험 결과는 해당 도서들이 질의와 정확한 단어 일치 없이도 의미적으로 유사함을 반영하며, 텍스트 내 표현이 직접적으로 등장하지 않음에도 불구하고, 감정적 분위기와 주제가 유사하여 높은 유사도 점수를 보였다. 이러한 차이가 발생하는 근본적인 이유는 두 접근 방식의 데이터 표현 방식 차이에 있다. 키워드 검색은 텍스트를 단순 문자열로 처리하지만 시맨틱 검색은 텍스트를 벡터 공간의 위치 정보로 변환한다. 즉, 시맨틱 검색에서는 다음과 같은 관계가 성립한다. "슬픈 사랑 이야기" ≈ "감정적으로 깊은 로맨스"

이처럼 표현은 다르지만 의미가 유사한 문장들이 벡터 공간에서는 가까운 위치에 존재하게 되며, 이를 통해 더 자연스러운 검색 결과를 얻을 수 있다. 그러나 시맨틱 검색 역시 완벽하지는 않다.

예를 들어 질의가 너무 짧은 경우 (“재밌는 책”) → 의미 정보 부족으로 정확도 감소하고, 데이터셋에 해당 주제가 부족한 경우 → 유사도 기반 추천 한계 발생한다.

결론적으로 실험 결과, 시맨틱 검색은 키워드 기반 검색에 비해 더 유연하고 의미 중심적인 결과를 제공함을 확인할 수 있었다. 특히 감정, 분위기, 주제와 같은 추상적인 개념을 포함한 질의에서 그 차이가 더욱 뚜렷하게 나타났다. 이는 텍스트를 벡터 공간으로 변환하고, 코사인 유사도를 통해 의미적 거리를 계산하는 접근 방식이 실제 문제 해결에 효과적으로 적용될 수 있음을 보여준다.

성공 사례로는 감정 기반 질의에서 높은 성능을 나타냈고, 다양한 표현에도 의미적으로 유사한 결과 반환했다.

실패 사례로는 매우 짧은 질의 (예: “재밌는 책”)에서 정확도가 낮았으며, 특정 전문 분야 질의에서 데이터 부족 문제가 발생했다.

프로젝트를 수행하며 배운점은 임베딩 기반 AI 시스템을 처음 접한 상태에서 시작되었기 때문에, 초기에는 개념 자체가 다소 추상적으로 느껴졌다. 특히 “텍스트를 숫자 벡터로 변환한다”는 개념이나, “검색과 분류가 같은 문제이다”라는 설명은 이론적으로는 이해가 잘 되지 않았다. 하지만 실제로 코드를 작성하고 실험을 진행하면서 이러한 개념들이 점차 구체적으로 이해되기 시작했다. 예를 들어, 사용자의 입력 문장을 임베딩으로 변환하고, 각 도서와의 코사인 유사도를 계산하여 가장 가까운 결과를 찾는 과정을 직접 구현하면서, 이것이 결국 벡터 공간에서 가장 가까운 점을 찾는 문제라는 것을 체감할 수 있었다. 특히 인상적이었던 부분은 키워드 기반 검색과 시맨틱 검색의 차이를 실험을 통해 확인한 경험이다. 단순히 단어가 일치하지 않아도 의미적으로 유사한 도서가 추천되는 결과를 보면서, 기존 검색 방식과는 전혀 다른 접근 방식이라는 것을 실감할 수 있었다. 이를 통해 자연어 처리에서 “의미”를 다루는 것이 얼마나 중요한지 이해하게 되었다. 또한 코사인 유사도를 직접 구현하고, 이를 라이브러리 결과와 비교하여 검증하는 과정에서도 많은 것을 배울 수 있었다. 단순히 함수를 사용하는 것이 아니라, 수식이 실제 코드에서 어떻게 계산되는지를 확인하면서 수학적 개념과 프로그래밍이 어떻게 연결되는지를 경험할 수 있었다. 한계점으로는 기본적인 개념을 구현하는 데 초점을 맞추었기 때문에 데이터셋의 규모가 충분히 크지 않다는 점이다. 제한된 개수의 도서 데이터만으로는 다양한 질의에 대해 항상 정확한 결과를 제공하기 어렵다. 특히 특정 주제나 장르에 대한 데이터가 부족한 경우, 추천 결과의 품질이 낮아지는 것을 확인할 수 있었다. 다음으로는 사용자의 질의 문장에 따라 결과의 정확도가 크게 달라진다는 점이다. 예를 들어 “재밌는 책”과 같이 추상적이고 짧은 질의의 경우, 의미 정보가 부족하여 유사도 계산이 정확하게 이루어지지 않는 문제가 발생하였다. 마지막으로 gpt-4.1-mini 모델을 활용한 텍스트 확장 과정은 임베딩 품질을 높이는 데 도움이 되었지만, 동시에 API 호출 횟수가 증가하여 시간과 비용 측면에서 비효율적인 부분이 존재하였다.

앞으로의 개선 방향으로는 다음과 같은 점들을 고려할 수 있다.
첫째, 더 다양한 도서 데이터를 확보하여 데이터셋의 규모를 확장하는 것이 필요하다. 데이터가

많아질수록 임베딩 공간에서의 분포가 더 정교해지고, 추천 성능도 향상될 것으로 예상된다.
둘째, 사용자 입력을 보다 효과적으로 처리하기 위한 방법을 추가할 수 있다. 예를 들어, 질의를 자동으로 보완하거나 구체화하는 기능을 추가한다면, 보다 정확한 추천 결과를 얻을 수 있을 것이다.
셋째, 현재는 단순한 리스트 형태의 추천 결과만 제공하지만, 향후에는 사용자 피드백을 반영하는 개인화 추천 시스템으로 확장할 수 있다. 이를 통해 사용자별로 더욱 맞춤화된 결과를 제공할 수 있을 것이다.
넷째, 현재는 단순 파일 기반으로 임베딩을 저장하고 있지만, 실제 서비스 수준에서는 벡터 데이터베이스를 활용하여 보다 빠르고 효율적인 검색 시스템을 구축할 수 있을 것이다.
마지막으로는 실제 수정사항을 보며 마치겠다. Json 파일에 생성된 책이 중복되어 나오는 경우가 있었는데 이 부분을 해결하는 과정이었다. 타이틀의 제목을 중복체크하고 프롬프트를 추가적으로 작성하였으며, 일부 코드를 수정하여, 프로젝트를 조금 더 완성도 있게 만든 경험이다.

```
def generate_books(n=50):
    books = []
    titles = set()  # 제목 중복 체크

    prompt = """
    책 데이터를 JSON 형식으로 만들어줘.
    반드시 아래 형식만 사용:

    {
      "title": "책 제목",
      "description": "책 설명",
      "review": "리뷰"
    }

    다른 말 절대 하지마.
    절대 중복되지 않게 만들어줘.
    리뷰에 대해서는 인물이나 감상을 더 풍부하게 표현해줘.
    """

    while len(books) < n:
        res = client.chat.completions.create(
            model="gpt-4.1-mini",
            messages=[{"role": "user", "content": prompt}]
        )

        try:
            data = json.loads(res.choices[0].message.content)

            ####### 중복 수정 전 코드 #######
            #if all(k in data for k in ["title", "description", "review"]):
            #books.append(data)

            ####### 중복 수정 후 코드 #######
            if not all(k in data for k in ["title", "description", "review"]):
                continue

            title = data["title"]

            if title in titles:
                continue

            titles.add(title)
            books.append(data)
            ##############################
        except:
            continue

    return books
```